In [1]:
```
import numpy as np
import pandas as pd
```

In [2]:
```
data = {
    'Country': ['France', 'Spain', 'Germany', 'Spain', 'Germany',
                'France', 'Spain', 'France', 'France', 'France'],
    'Age': [44, 27, 30, 38, 40, 35, 38, 48, 50, 37],
    'Salary': [72000, 48000, 54000, 61000, 63778,
               58000, 52000, 79000, 83000, 67000],
    'Purchased': ['No', 'Yes', 'No', 'No', 'Yes',
                  'Yes', 'No', 'Yes', 'No', 'Yes']
}

df = pd.DataFrame(data)

df.to_csv('pre-process_datasample.csv', index=False)

print("pre-process_datasample.csv created successfully")
```

```
pre-process_datasample.csv created successfully
```

In [3]:
```
df = pd.read_csv("pre-process_datasample.csv")
df
```

Out[3]:

	Country	Age	Salary	Purchased
0	France	44	72000	No
1	Spain	27	48000	Yes
2	Germany	30	54000	No
3	Spain	38	61000	No
4	Germany	40	63778	Yes
5	France	35	58000	Yes
6	Spain	38	52000	No
7	France	48	79000	Yes
8	France	50	83000	No
9	France	37	67000	Yes

In [4]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype
---  ------     --------------  -----
 0   Country    10 non-null     str
 1   Age        10 non-null     int64
 2   Salary     10 non-null     int64
 3   Purchased  10 non-null     str
dtypes: int64(2), str(2)
memory usage: 452.0 bytes
```

In [5]:
```
df.Country.mode()
```

Out[5]:
```
0    France
Name: Country, dtype: str
```

In [6]:
```
df.Country.mode()[0]
```

Out[6]:
```
'France'
```

In [7]:
```
type(df.Country.mode())
```

Out[7]:
```
pandas.Series
```

In [8]:
```
df.Country.fillna(df.Country.mode()[0], inplace=True)
df
```

```
C:\Users\DELL\AppData\Local\Temp\ipykernel_15304\1453460914.py:1: ChainedAssignmentE
rror: A value is being set on a copy of a DataFrame or Series through chained assign
ment using an inplace method.
Such inplace method never works to update the original DataFrame or Series, because
the intermediate object on which we are setting values always behaves as a copy (due
to Copy-on-Write).

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method
({col: value}, inplace=True)' instead, to perform the operation inplace on the origi
nal object, or try to avoid an inplace operation using 'df[col] = df[col].method(val
ue)'.

See the documentation for a more detailed explanation: https://pandas.pydata.org/pan
das-docs/stable/user_guide/copy_on_write.html
  df.Country.fillna(df.Country.mode()[0], inplace=True)
```

Out[8]:

	Country	Age	Salary	Purchased
0	France	44	72000	No
1	Spain	27	48000	Yes
2	Germany	30	54000	No
3	Spain	38	61000	No
4	Germany	40	63778	Yes
5	France	35	58000	Yes
6	Spain	38	52000	No
7	France	48	79000	Yes
8	France	50	83000	No
9	France	37	67000	Yes

In [9]:
```
df.Age.fillna(df.Age.median(), inplace=True)
df
```

```
C:\Users\DELL\AppData\Local\Temp\ipykernel_15304\3656844748.py:1: ChainedAssignmentE
rror: A value is being set on a copy of a DataFrame or Series through chained assign
ment using an inplace method.
Such inplace method never works to update the original DataFrame or Series, because
the intermediate object on which we are setting values always behaves as a copy (due
to Copy-on-Write).

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method
({col: value}, inplace=True)' instead, to perform the operation inplace on the origi
nal object, or try to avoid an inplace operation using 'df[col] = df[col].method(val
ue)'.

See the documentation for a more detailed explanation: https://pandas.pydata.org/pan
das-docs/stable/user_guide/copy_on_write.html
  df.Age.fillna(df.Age.median(), inplace=True)
```

Out[9]:

	Country	Age	Salary	Purchased
0	France	44	72000	No
1	Spain	27	48000	Yes
2	Germany	30	54000	No
3	Spain	38	61000	No
4	Germany	40	63778	Yes
5	France	35	58000	Yes
6	Spain	38	52000	No
7	France	48	79000	Yes
8	France	50	83000	No
9	France	37	67000	Yes

In [10]:
```
df.Salary.fillna(round(df.Salary.mean()), inplace=True)
df
```

```
C:\Users\DELL\AppData\Local\Temp\ipykernel_15304\950604936.py:1: ChainedAssignmentEr
ror: A value is being set on a copy of a DataFrame or Series through chained assignm
ent using an inplace method.
Such inplace method never works to update the original DataFrame or Series, because
the intermediate object on which we are setting values always behaves as a copy (due
to Copy-on-Write).

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method
({col: value}, inplace=True)' instead, to perform the operation inplace on the origi
nal object, or try to avoid an inplace operation using 'df[col] = df[col].method(val
ue)'.

See the documentation for a more detailed explanation: https://pandas.pydata.org/pan
das-docs/stable/user_guide/copy_on_write.html
  df.Salary.fillna(round(df.Salary.mean()), inplace=True)
```

Out[10]:

	Country	Age	Salary	Purchased
0	France	44	72000	No
1	Spain	27	48000	Yes
2	Germany	30	54000	No
3	Spain	38	61000	No
4	Germany	40	63778	Yes
5	France	35	58000	Yes
6	Spain	38	52000	No
7	France	48	79000	Yes
8	France	50	83000	No
9	France	37	67000	Yes

In [11]:
```
pd.get_dummies(df.Country)
```

Out[11]:

	France	Germany	Spain
0	True	False	False
1	False	False	True
2	False	True	False
3	False	False	True
4	False	True	False
5	True	False	False
6	False	False	True
7	True	False	False
8	True	False	False
9	True	False	False

In [12]:
```
updated_dataset = pd.concat(
    [pd.get_dummies(df.Country), df.iloc[:, [1, 2, 3]]],
    axis=1
)

updated_dataset
```

Out[12]:

	France	Germany	Spain	Age	Salary	Purchased
0	True	False	False	44	72000	No
1	False	False	True	27	48000	Yes
2	False	True	False	30	54000	No
3	False	False	True	38	61000	No
4	False	True	False	40	63778	Yes
5	True	False	False	35	58000	Yes
6	False	False	True	38	52000	No
7	True	False	False	48	79000	Yes
8	True	False	False	50	83000	No
9	True	False	False	37	67000	Yes

In [13]:
```
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype 
---  ------     --------------  ----- 
 0   Country    10 non-null     str   
 1   Age        10 non-null     int64 
 2   Salary     10 non-null     int64 
 3   Purchased  10 non-null     str   
dtypes: int64(2), str(2)
memory usage: 452.0 bytes
```

In [14]:
```
updated_dataset.Purchased.replace(
    ['No', 'Yes'],
    [0, 1],
    inplace=True
)
```

```
C:\Users\DELL\AppData\Local\Temp\ipykernel_15304\2609612589.py:1: ChainedAssignmentE
rror: A value is being set on a copy of a DataFrame or Series through chained assign
ment using an inplace method.
Such inplace method never works to update the original DataFrame or Series, because
the intermediate object on which we are setting values always behaves as a copy (due
to Copy-on-Write).

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method
({col: value}, inplace=True)' instead, to perform the operation inplace on the origi
nal object, or try to avoid an inplace operation using 'df[col] = df[col].method(val
ue)'.

See the documentation for a more detailed explanation: https://pandas.pydata.org/pan
das-docs/stable/user_guide/copy_on_write.html
  updated_dataset.Purchased.replace(
```

Out[14]:
```
0    0
1    1
2    0
3    0
4    1
5    1
6    0
7    1
8    0
9    1
Name: Purchased, dtype: object
```

In [15]:
```
updated_dataset
```

Out[15]:

	France	Germany	Spain	Age	Salary	Purchased
0	True	False	False	44	72000	No
1	False	False	True	27	48000	Yes
2	False	True	False	30	54000	No
3	False	False	True	38	61000	No
4	False	True	False	40	63778	Yes
5	True	False	False	35	58000	Yes
6	False	False	True	38	52000	No
7	True	False	False	48	79000	Yes
8	True	False	False	50	83000	No
9	True	False	False	37	67000	Yes

In [ ]: